

ER Diagrams

Drawn as entities being rectangles with their attributes inside the rectangle. Note that the PK is typically underlined. Relations between entities are marked as diamonds. The connectors between a diamond and the entity can have different shapes / forms. If there are 2 connecting lines that means total participation, meaning every entity is part of this relation. Arrows from the relation to the entity indicates the cardinality of the relation, with presence of an arrow being ___ to one. In other words, for an advisor database, courses could only be assigned one teacher so there would be an arrow from teaches to the teacher entity.

Weak entity sets are entities who are not uniquely identifiable on thier own and rely on their relation to another entity in order to identify them.

Note that relations are allowed to have attributes (for teaches could be the semester / year and room) which is modeled by a dotted line coming from the relation to to a new rectangle of the extended attributes.

Subclasses are very similar to ideas of object oriented programming where the subclasses inherit the parent attributes. This is modeled as a hierarchical relation with the parent class at the top (standard entity notation) which then points to a triangle labeled “isa” which points to all the subclasses.

Aggregation methods allow a relation to treat another relation as an entity. This is modeled by a squircle surrounding thte relation and its entities.

SQL Queries

```
SELECT Attributes, Agg(A_i), Distinct(Grouping Attributes)
FROM Tables
WHERE Conditions
GROUP BY Grouping Attributes
HAVING Grouping Conditions
```

Some aggregate functions include:

- Sum
- Count
- Max
- Min
- Avg

Note for grouping SQL queries, the select clause must contain only attributes that are either within an aggregate function or a part of the grouping attributes.

Sub Queries

```
SELECT Attributes
FROM Tables
WHERE Conditions
    AND Sub Attributes IN / NOT IN (
        SELECT Sub Attributes
        FROM Tables
        WHERE BLAH
    )
```

We can also replace the IN / NOT IN with a > ALL or > SOME quantifier. There's also the EXISTS where a query could be

```
SELECT Attributes
FROM Tables
WHERE Conditions
    AND EXISTS / NOT EXISTS (
        SELECT Sub Attributes
        FROM Tables
        WHERE BLAH
    )
```

One that will most likely be on the exam is a “find all students (id) who have taken every biology course”:

```
SELECT S.id
FROM Students as S
WHERE NOT EXISTS (
    (SELECT C.id
     FROM Courses as C
     WHERE C.dpt = "Biology")
    EXCEPT
    (SELECT E.CourseId
     FROM Enrollment as E
     WHERE E.id = S.id)
)
```

You could easily alter the above query to have taken ONLY biology courses by in the outer query filtering by students who have taken a biology course and the inner query looking for if the student has taken a course that is not biology.

Relational Algebra

Mostly just think in terms of SQL and translate into RA. Below are the general tools or functions for RA:

- $\pi_{Attributes}$ which is projection onto the attributes in the subscript
- $\sigma_{Conditions}$ which is like the WHERE clause in SQL
- $\bowtie_{JoinConditions}$ which is the join operator.
- $=\bowtie_{JoinConditions}$ which is left join which is left join (right join is other side and outer is both)
- $\cup$ Union (attributes should be same thing)
- $\cap$ Union (attributes need to be same thing)
- $-$ Set subtraction (useful for exists / not exists)
- $\times$ cross product
- $\{A, B\}/B$ This returns the values of A that have an entry for every value in B
- $groupattributes \gamma_{F(Attributes)}$ group/aggregate functions where F are aggregate functions
- $\rho(New, Old)$ rename operation

Normalization

Identifying (Candidate) Keys

Super keys SK are defined for a relation R as $SK \subseteq R$ such that $SK \rightarrow R$. In other words, they are a set of attributes of R such that knowing these attributes for any tuple you are able to determine the rest of the attributes of the tuple.

A key $K \in SK$ is defined as $\forall sk \in SK, |K| \leq |sk|$

Normal Forms

For FD's $X \rightarrow Y$

BCNF:

1. $Y \subseteq X$
2. $X \in SK$

3NF:

1. BCNF
2. $\forall A \in Y, A \in K$

Lossless

A decomp into R_1, R_2 is lossless iff:

$$(R_1 \cap R_2) \rightarrow R_1 \vee (R_1 \cap R_2) \rightarrow R_2$$

In other words lossless if there exists a key in their intersection for at least one of the relations.

Decompositions

```
while S in DECOMP and S not in BCNF:
    find X -> Y in FD as F, where X not a SK of S

    DECOMP.pop(S)

    DECOMP.append(S_1=(XY, F))

    DECOMP.append(S_1=(S-(Y-X), FD-F))
```

In english, just remove the attributes for FK's that violate BCNF into a separate relation until ur bing chilling

Minimal Cover

Common sense test

1. Convert all FD's to have only one attributes on right
2. Eliminate Redundant Attributes From left
3. Transitive property this thing up !!!

Storage

We can think of storing entities down to their bytes, a singular entity most likely has multiple attributes of different sizes.

Size of an individual tuple is $\forall A_i \in R, \sum |a_i| : |a_i| \geq a \in A_i$. However if the storage structure also includes things like pointers then they should be added to this sum as well.

Number of records per page is $\lfloor \frac{|Page|}{|Tuple|} \rfloor$ and the number of pages to store a set of n tuples is $\lceil \frac{n|Tuples|}{|Page|} \rceil$ if records can be split across pages. Note sometimes there are page headers which should be subtracted from the size of the page.